

Voiland Food Pantry System

Bug Testing and Remediation Guide

Prepared for: Future Development Team

Project Type: Java Spring Boot Local Terminal System

Date: May 7, 2026

Purpose of This Document

This document identifies likely bugs, fragile systems, architectural risks, and reliability concerns discovered during a static review of the Voiland Food Pantry codebase.

The goal is to help the next development team:

- identify weak areas quickly,
- reproduce known problems,
- understand why problems occur,
- and prioritize future fixes.

This is not intended as criticism of the original team. The project includes substantial functionality and appears to have evolved rapidly during development. Many issues are common in large student projects built under deadline pressure.

Priority Levels

Priority	Meaning
CRITICAL	Security issue or major data corruption risk
HIGH	Likely to break under real use
MEDIUM	Causes instability or inconsistent behavior
LOW	Cleanup, maintainability, or usability issue

1. Inventory Race Condition / Multi-User Corruption

Priority: HIGH

Problem

Inventory quantity updates are not protected against simultaneous access.

The system:

1. reads the current inventory quantity,
2. subtracts one,
3. saves the updated value.

There is no locking or transaction protection.

If two users scan the same item at nearly the same time:

- both may read the same quantity,
 - both may write conflicting updates,
 - inventory counts become inaccurate.
-

Why This Matters

This issue may not appear during single-user demos, but becomes serious when:

- multiple terminals are added,
- the system is hosted online,
- or multiple volunteers use the system simultaneously.

This can cause:

- incorrect inventory counts,
 - inaccurate pantry analytics,
 - failed stock tracking.
-

Likely Location

Look in:

- InventoryController
- inventory checkout endpoints
- inventory service/repository update logic

Look for code patterns similar to:

```
item.setQuantity(item.getQuantity() - 1);  
repository.save(item);
```

How To Test

Test A: Simultaneous Scan Test

1. Set item quantity to 1.
2. Open two browser sessions.
3. Scan or submit the same item simultaneously.
4. Refresh inventory.

Expected bad behavior:

- both submissions succeed,
 - quantity becomes incorrect,
 - duplicate checkout may occur.
-

Potential Fixes

Better Fix

Use database transactions and locking.

Possible approaches:

- `@Transactional`
 - optimistic locking (`@Version`)
 - pessimistic locking
-

Minimum Fix

Re-check inventory quantity immediately before saving.

2. Scanner JSON Crash Vulnerability

Priority: HIGH

Problem

Scanner endpoints assume incoming JSON is always perfectly formatted.

Endpoints consume JSON like:

```
{"code": "12345"}
```

There is little or no malformed-input recovery.

If:

- the scanner sends partial input,
- malformed characters,
- duplicate carriage returns,
- or invalid payloads,

backend exceptions may occur.

This likely caused the previously reported JSON crashes.

Why This Matters

Barcode scanners behave differently depending on:

- model,
- USB mode,
- keyboard emulation settings,
- browser focus,
- timing.

Real hardware frequently produces malformed edge cases.

Likely Location

Look for:

```
@PostMapping(consumes = "application/json")
```

Especially:

- inventory checkout endpoints
- barcode validation endpoints

- scanner processing controllers
-

How To Test

Test A: Invalid JSON

Send malformed requests:

```
{"code":}
```

or:

```
invalid
```

Expected bad behavior:

- server error,
 - stack trace,
 - frontend freeze,
 - broken workflow.
-

Test B: Rapid Repeated Scans

Scan the same item repeatedly very quickly.

Expected bad behavior:

- duplicate requests,
 - desynchronized frontend,
 - failed inventory updates.
-

Test C: Scanner Disconnect Mid-Workflow

1. Start scanning.
2. Disconnect scanner.
3. Attempt manual entry.

Expected bad behavior:

- input no longer works,
- frontend stuck,

- invalid state.
-

Potential Fixes

Backend

Add:

- centralized exception handling,
- request validation,
- graceful malformed JSON recovery.

Use:

@ControllerAdvice

Frontend

Add:

- retry handling,
 - scanner timeout handling,
 - input reset buttons,
 - duplicate scan prevention.
-

3. Inventory Endpoints Missing Authorization

Priority: HIGH

Problem

Some inventory-related endpoints appear publicly accessible.

SecurityConfig allows broad inventory access patterns.

Certain controller methods do not independently verify authentication.

Potentially exposed:

- inventory checkout,
- barcode validation,

- inventory reads.
-

Why This Matters

If hosted online later:

- unauthorized users may manipulate inventory,
 - inventory information may leak,
 - checkout requests could be forged.
-

Likely Location

Look in:

- SecurityConfig
- inventory controller mappings

Search for:

`.permitAll()`

and:

`"/inventory/**"`

How To Test

Test A: Access Without Login

1. Log out.
2. Directly navigate to inventory endpoints.
3. Attempt API requests manually.

Expected bad behavior:

- access still succeeds,
 - checkout still functions.
-

Potential Fixes

Require authentication for all inventory endpoints.

Use:

`.authenticated()`

and role-based authorization.

4. Weak Admin Credentials

Priority: HIGH

Problem

Known default credentials:

- admin
 - password123
-

Why This Matters

If unchanged:

- anyone familiar with the project can access admin functions.

This is especially dangerous if:

- the database is exposed,
 - remote hosting is added,
 - backups are shared.
-

Likely Location

Look for:

- seeded admin creation,
 - startup initialization,
 - application bootstrap logic.
-

How To Test

Attempt login using known defaults.

Potential Fixes

- force password reset on first launch,
 - remove hardcoded defaults,
 - use environment configuration.
-

5. Hardcoded Database Credentials

Priority: CRITICAL

Problem

Database credentials are stored directly in source code.

Why This Matters

This is unsafe because:

- credentials can leak through Git repositories,
 - future developers inherit root access,
 - online deployment becomes dangerous.
-

Likely Location

Look in:

`application.properties`

or:

`application.yml`

How To Test

Open configuration files and verify:

```
spring.datasource.username=  
spring.datasource.password=
```

Potential Fixes

Move credentials into:

- environment variables,
- .env files,
- deployment secrets.

Never commit credentials into repositories.

6. CSRF Protection Disabled

Priority: HIGH

Problem

Spring Security CSRF protection is disabled globally.

Why This Matters

This may not matter much locally, but becomes dangerous online.

Attackers could potentially:

- forge requests,
 - trigger inventory updates,
 - abuse authenticated sessions.
-

Likely Location

Look in:

SecurityConfig

Search for:

```
.csrf().disable()
```

How To Test

Attempt cross-origin requests or form submissions.

Potential Fixes

Enable CSRF for browser-based operations.

Use Spring Security defaults where possible.

7. Browser Navigation and Session Problems

Priority: MEDIUM

Problem

The frontend relies heavily on:

- redirects,
- browser state,
- session variables.

Late addition of back buttons suggests navigation problems were discovered late.

Why This Matters

Users may experience:

- stale pages,
 - returning after logout,
 - duplicate submissions,
 - broken scanner focus.
-

Likely Location

Look in:

- frontend templates,
 - login flows,
 - session handling code,
 - redirect logic.
-

How To Test

Test A: Browser Back Button

1. Log in.
2. Navigate through workflows.
3. Log out.
4. Press browser back.

Expected bad behavior:

- access to stale pages,
 - broken session state.
-

Test B: Multiple Tabs

1. Open multiple tabs.
2. Perform different workflows.

Expected bad behavior:

- session confusion,
 - incorrect data shown.
-

Potential Fixes

- stronger session invalidation,
- redirect cleanup,
- frontend state management,

- cache-control headers.
-

8. Limited Error Handling

Priority: HIGH

Problem

The application relies heavily on:

- RuntimeException,
- console debugging,
- direct Optional handling.

There is little centralized recovery behavior.

Why This Matters

Unexpected input may:

- crash workflows,
 - expose stack traces,
 - leave inconsistent application state.
-

Likely Location

Look across controllers for:

`throw new RuntimeException`

or missing try/catch handling.

How To Test

Submit:

- empty fields,
- invalid IDs,

- malformed requests,
- oversized values.

Expected bad behavior:

- server errors,
 - blank pages,
 - failed redirects.
-

Potential Fixes

Add:

- centralized exception handlers,
 - validation responses,
 - structured error messages.
-

9. Duplicate Data Risks

Priority: MEDIUM

Problem

Duplicate prevention relies mostly on application logic instead of database constraints.

Possible duplicates:

- volunteers,
 - students,
 - inventory entries,
 - repeated scans.
-

Why This Matters

Application-side checks alone are fragile.

Race conditions and unexpected requests may bypass them.

Likely Location

Look at:

- entity annotations,
 - database schema,
 - repository validation logic.
-

How To Test

Submit the same registration or scan repeatedly.

Expected bad behavior:

- duplicate records,
 - inconsistent reporting.
-

Potential Fixes

Add:

- unique constraints,
 - database indexes,
 - validation layers.
-

10. Report and PDF Export Fragility

Priority: MEDIUM

Problem

Report generation appears minimally protected against:

- null values,
 - empty datasets,
 - malformed records.
-

Why This Matters

Reports may:

- fail unexpectedly,
 - generate incomplete PDFs,
 - freeze with larger datasets.
-

Likely Location

Look in:

- reporting controllers,
 - PDF generation utilities,
 - analytics aggregation code.
-

How To Test

Test A: Empty Database

Run reports with:

- no visits,
- no inventory,
- no volunteers.

Expected bad behavior:

- crashes,
 - broken exports.
-

Test B: Large Dataset

Populate large inventory and visit history.

Expected bad behavior:

- slowdown,
- timeouts,
- memory spikes.

Potential Fixes

Add:

- null checking,
 - pagination,
 - query optimization,
 - defensive PDF generation.
-

11. Logging Sensitive Data

Priority: MEDIUM

Problem

Debug logging prints user identifiers and workflow details to the console.

Why This Matters

Sensitive information may appear in:

- terminal windows,
 - logs,
 - screenshots,
 - shared debug output.
-

Likely Location

Search for:

`System.out.println`

How To Test

Run normal workflows and inspect terminal output.

Potential Fixes

Use:

- proper logging framework,
 - sanitized logs,
 - configurable debug levels.
-

Recommendations for Future Development Team

Highest Priority Improvements

1. Fix inventory concurrency problems
 2. Secure inventory endpoints
 3. Remove hardcoded credentials
 4. Improve scanner input validation
 5. Add centralized exception handling
 6. Harden session management
 7. Add database constraints
-

Recommendation Before Online Hosting

The current system should NOT be hosted publicly without major redesign and security hardening.

Required before online deployment:

- secure authentication architecture,
- proper authorization,
- concurrency-safe inventory updates,
- secure credential management,
- CSRF protection,

- production-grade logging and monitoring,
 - resilient scanner/input handling.
-

Final Notes

This project demonstrates substantial functionality and integration effort, especially considering:

- hardware integration,
- reporting,
- inventory tracking,
- volunteer workflows,
- and browser-based operation.

However, many current issues are typical of rapidly expanded academic projects and should be treated as expected technical debt rather than isolated mistakes.

The next development team should prioritize:

- stabilization,
- cleanup,
- testing,
- and architectural hardening

before expanding feature scope further.

This analysis was made with ChatGPT by analyzing the final upload of the dev team. Note that this upload was hours before the final presentation, so no further repairs were possible at that point.

The program is functional and reasonably secure for offline terminal use, but it is fragile. Testing and revisions for stability and security must be completed before any online deployment. Revisions for more robust code with better error handling and security are preferable for offline deployment.